

Hoja de Ejercicios Complementarios: Bloque 2

Ficheros Binarios, Acceso Posicional Directo (RAF) y Serialización de Objetos

MÓDULO PROFESIONAL	CICLO Y UNIDAD	MODALIDAD / ENTORNO	DESTINATARIO / CARÁCTER
Acceso a Datos (0486)	DAM 2º Curso · UT1	Presencial / Distancia · Java 21	Alumnado · Consolidación Técnica

ALINEACIÓN CURRICULAR OFICIAL (REAL DECRETO 455/2010 · CURRÍCULO ASTURIAS)

RA1: Manejo de Ficheros

CE1.c (Ficheros Binarios y Acceso Aleatorio)

CE1.d (Serialización de Grafos de Objetos)

Este bloque se centra en la persistencia binaria de alto rendimiento y bajo nivel: tipado estricto con `DataStreams` y captura de `EOFException`, manipulación física del cursor en disco mediante registros de longitud fija con `RandomAccessFile` (cálculo de *offsets* y modificaciones *in-situ*), y serialización de objetos en modo *append* seguro resolviendo el problema de la doble cabecera (`MiObjectOutputStream`).

🚩 Instrucciones de Trabajo y Buenas Prácticas de Persistencia Binaria

En los ficheros binarios no existen delimitadores de texto (ni saltos de línea `\n` ni punto y coma `;`). La lectura debe ser el espejo exacto de la secuencia de tipos primitivos escritos. Recuerda siempre que al serializar objetos debes declarar explícitamente `serialVersionUID` y utilizar `transient` en atributos sensibles.

Contexto Empresarial:

Una compañía de logística urgente equipa sus furgonetas con un microcontrolador que almacena en tiempo real tramas GPS en un fichero binario local (`datos/flota_gps.dat`). Dado que la memoria flash del dispositivo es sumamente limitada, está prohibido el uso de formatos de texto redundantes (como CSV o JSON).

Requerimientos Técnicos e Instrucciones:

1. **Escritura de Tramas Tipadas:** Implementa el método:

```
public static void registrarPosicion(File archivo, int idVehiculo, String matricula,
                                   double latitud, double longitud, double velocidad, boolean enRuta)
```

Utiliza `DataOutputStream` envolviendo `FileOutputStream(archivo, true)`. Escribe los campos en este orden estricto: `writeInt`, `writeUTF`, `writeDouble`, `writeDouble`, `writeDouble` y `writeBoolean`.

2. **Lectura Secuencial y Control de Fin de Fichero:** Implementa el método:

```
public static void generarInformeExcesosVelocidad(File archivo, double limiteVelocidad)
```

Abre el archivo con `DataInputStream`. Ejecuta un bucle `while (true)` leyendo secuencialmente en el mismo orden exacto. **Captura de forma limpia** `EOFException` como señal normal de final de archivo para salir del bucle.

3. **Detección de Infracciones:** Muestra por consola las tramas cuya velocidad supere el umbral indicado (120.0 km/h) indicando matrícula, coordenadas geográficas y velocidad registrada.

Trampa Típica: Detección de Fin de Archivo en `DataInputStream`

`DataInputStream.readInt()` o `readDouble()` **no devuelven** `-1` al llegar al final del archivo. La única forma reglamentaria de detectar el fin de datos es capturar la excepción `EOFException`. Jamás consideres esta excepción como un error de ejecución.

Salida Esperada en Consola:

```
[INFO] Registrando 4 tramas telemétricas en 'datos/flota_gps.dat'...
[AUDITORÍA] Analizando excesos de velocidad (Límite: 120.0 km/h):
-----
¡ALERTA! Vehículo ID: 104 | Matrícula: 5544-HJK | Vel: 134,80 km/h
        Posición: (43.535200, -5.924800) | En Ruta: true
-----
[FIN] Fin de fichero alcanzado (EOFException capturada limpiamente).
```

Ejercicio 2: Almacén de Acceso Directo con Registros Fijos de 44 Bytes

(AlmacenRAF.java)

NIVEL
INTERMEDIO

Contexto Empresarial:

Un centro logístico gestiona un fichero binario `datos/articulos.dat` con 20.000 referencias. Cuando llega un camión de aprovisionamiento, se debe actualizar el stock del producto de inmediato. Reescribir todo el fichero para modificar unos pocos bytes resulta inaceptable; se debe utilizar acceso aleatorio posicional mediante `RandomAccessFile`.

Estructura Binaria de Registro Fijo (44 Bytes exactos):

Campo	Tipo en Java	Instrucción de Escritura	Tamaño Físico	Offset Relativo
ID Artículo	<code>int</code>	<code>raf.writeInt(id)</code>	4 bytes	0
Código Barras	<code>String</code> (fijo 8 chars)	<code>raf.writeChars(codigo8)</code>	16 bytes (\$8 \times 2\$)	+4
Nombre Producto	<code>String</code> (fijo 10 chars)	<code>raf.writeChars(nombre10)</code>	20 bytes (\$10 \times 2\$)	+20
Stock	<code>int</code>	<code>raf.writeInt(stock)</code>	4 bytes	+40

Métodos Requeridos en AlmacenRAF :

- `insertarArticulo(int pos, int id, String cod, String nom, int stock)` : Calcula `raf.seek(pos * 44)` . Emplea un método auxiliar para ajustar los textos a la longitud exacta rellenando con espacios a la derecha o truncando.
- `actualizarStock(int pos, int nuevoStock)` : Realiza un salto quirúrgico directo a la posición exacta del campo stock: `raf.seek((pos * 44) + 40)` y sobrescribe **únicamente** los 4 bytes con `raf.writeInt(nuevoStock)` .
- `consultarArticulo(int pos)` : Posiciona el puntero en `pos * 44` , lee el entero, lee los 8 caracteres del código, los 10 caracteres del nombre (con bucles de `readChar()`) y los 4 bytes de stock.
- `obtenerTotalArticulos()` : Retorna `raf.length() / 44` .
- `truncarAlmacen(int maxRegistros)` : Trunca físicamente el archivo en disco mediante `raf.setLength(maxRegistros * 44)` .

⚠ Advertencia sobre writeChars vs writeUTF

En registros de tamaño constante, **no utilices** `writeUTF` porque inserta una cabecera variable de 2 bytes con la longitud del texto. Se debe utilizar `writeChars()` asegurando que la cadena tenga exactamente los caracteres fijados en el diseño.

✓ Salida Esperada en Consola:

```
[INFO] Insertando 3 artículos en registros fijos de 44 bytes...
[INFO] Total artículos en disco: 3 (Tamaño archivo: 132 bytes)
[CONSULTA] Posición 1: ID: 102 | Código: 'ES889900' | Nombre: 'Teclado ' | Stock: 15
[OPERACIÓN] Actualizando stock de Posición 1 a 40 unidades (salto directo a byte 84)...
[VERIFICACIÓN] Posición 1 tras actualización: Stock: 40 unidades.
[TRUNCADO] Reduciendo almacén a 2 artículos... Tamaño resultante: 88 bytes.
```

Ejercicio 3: Auditoría Bancaria con Append Seguro y `transient`

(`AuditoriaBancaria.java`)

NIVEL
AVANZADO

Contexto Empresarial:

Una entidad bancaria audita transacciones financieras persistiendo objetos de negocio en un archivo centralizado `datos/auditoria_banco.ser`. Cada transacción debe anexarse de manera inmediata al archivo físico. Además, el protocolo PCI-DSS prohíbe terminantemente almacenar en el disco claves o PINes de autorización.

Requerimientos Técnicos del Sistema:

1. Clase de Entidad `Transaccion` :

- Implementa `Serializable` con `private static final long serialVersionUID = 1L;`.
- Atributos: `long id`, `String cuentaOrigen`, `String cuentaDestino`, `double importe`, `LocalDateTime fecha`.
- **Campo de Seguridad:** `private transient String pinAutorizacion;` El PIN se utiliza en memoria para validar la operación, pero no debe escribirse en disco.

2. Clase Utilitaria `MiObjectOutputStream` : Hereda de `ObjectOutputStream` y sobrescribe `protected void writeStreamHeader() throws IOException {}` dejándolo completamente vacío, evitando así la duplicación de la cabecera mágica (`0xACED`).

3. Método de Persistencia Incremental Inteligente:

```
public static void persistirTransaccion(File archivo, Transaccion t)
```

Si el archivo no existe o su tamaño es `0`, instancia `new ObjectOutputStream(new FileOutputStream(archivo, true))`. Si el archivo ya existe y tiene contenido, instancia `new MiObjectOutputStream(new FileOutputStream(archivo, true))`. Asegura siempre la llamada a `oos.flush()`.

4. Lectura y Verificación de Integridad: Recupera todos los objetos en un bucle continuo hasta `EOFException`. Verifica que se deserializan todas las transacciones sin que surja `StreamCorruptedException` y comprueba que `pinAutorizacion` se recupera como `null`.

La Anomalía de la Doble Cabecera en Serialización

Si abres un `ObjectOutputStream` sobre un fichero existente en modo append sin anular su cabecera con `writeStreamHeader()`, Java escribirá una cabecera de 4 bytes intermedia. Al leer el segundo objeto, el `ObjectInputStream` esperará un descriptor de objeto y encontrará la cabecera, lanzando la clásica excepción: `java.io.StreamCorruptedException: invalid type code: AC`.

Salida Esperada en Consola:

```
[TRANSACCIÓN 1] Creada con PIN '4921' -> Persistida (nuevo archivo con cabecera estándar).  
[TRANSACCIÓN 2] Creada con PIN '1184' -> Anexada (usando MiObjectOutputStream sin cabecera).  
[TRANSACCIÓN 3] Creada con PIN '9932' -> Anexada (usando MiObjectOutputStream sin cabecera).
```

```
=== LECTURA DEL LIBRO MAYOR DE AUDITORÍA ===
```

```
Tx ID: 1001 | De: ES76-2048... | A: ES91-0049... | Importe: 1250,00 € | PIN: null
```

```
Tx ID: 1002 | De: ES33-0081... | A: ES12-3344... | Importe: 340,50 € | PIN: null
```

```
Tx ID: 1003 | De: ES99-1122... | A: ES77-8899... | Importe: 5200,00 € | PIN: null
```

```
=====
```

```
[VERIFICACIÓN] Todas las operaciones deserializadas con éxito sin corrupción de flujo.
```

```
[SEGURIDAD] Comprobado: Los PINes son 'null' (el modificador transient protegió los datos).
```